

# Reviewer Pack — Minimal Rank of Affine Quotient Sheaves and Circuit Monotone...

Entry 7d9763a91a56 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 01:48:49 UTC

## Minimal Rank of Affine Quotient Sheaves and Circuit Monotone Complexity

**Entry ID:** 7d9763a91a56 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 01:48:49 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic Geometry (specifically, sheaf theory) **Field B** (complexity object): Boolean Circuits

#### Statement:

For every  $d$ -regular graph  $G$ , the minimal rank of the affine quotient sheaf associated with its Tseitin formula  $\varphi_G$  is linearly correlated with its monotone circuit complexity  $m(\varphi_G)$ , such that  $\min\_rank(\varphi_G) = \Theta(m(\varphi_G))$ .

#### Rationale (proposer's reasoning):

Sheaf theory provides a framework for understanding complex geometric objects by decomposing them into simpler components. By applying this to affine quotient sheaves, we can potentially uncover deeper structures in the Boolean circuit complexity of Tseitin formulas, which could lead to new insights or lower bounds in computational complexity.

**Taxonomy category:** AlgebraicGeometryBooleanCircuits (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** b320c03d4a4175eb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

For every  $d$ -regular graph  $G$  with  $n$  vertices, the minimal rank of its affine quotient sheaf  $\min\_rank(\varphi_G)$  is considered supported if there exists a linear relationship ( $R^2 \geq 0.9$ ) between  $\min\_rank(\varphi_G)$  and  $m(\varphi_G)$ , where  $m(\varphi_G)$  is the monotone circuit complexity, across at least 30 seeds. Falsified if any seed shows an  $R^2 < 0.5$  or if any graph has a  $\min\_rank(\varphi_G) / m(\varphi_G)$  ratio outside of  $[0.1, 10]$ .

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| NATURAL_PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 1.00       | SAFE             | SAFE             |
| KARP_LIPTON    | SAFE    | 1.00       | UNCERTAIN        | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "affine quotient sheaf" AND "Tseitin formula" AND "monotone circuit complexity" - "minimal rank sheaf theory" AND Boolean circuits - "algebraic geometry" AND Tseitin formula AND monotone complexity

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

### 5.1 Generated Python source

```
import random
from fractions import Fraction

def generate_d_regular_graph(n, d):
    if (n * d) % 2 != 0:
        return None
    edges = set()
    for _ in range(d * n // 2):
        u = random.randint(0, n - 1)
        v = random.randint(0, n - 1)
        while u == v or (u, v) in edges or (v, u) in edges:
            u = random.randint(0, n - 1)
            v = random.randint(0, n - 1)
        edges.add((u, v))
    return edges

def tseitin_formula(graph, n):
    literals = {f'x{i}': i for i in range(n)}
    clauses = []
    for u, v in graph:
        clauses.append([f'x{u}', f'~x{v}'])
        clauses.append([f'~x{u}', f'x{v}'])
        clauses.append([f'~x{u}', f'~x{v}', f'x{n + u + v}'])
    return literals, clauses

def min_rank_of_affine_quotient_sheaf(clauses):
    # Placeholder for actual implementation
    return len(clauses)

def monotone_circuit_complexity(literals, clauses):
    # Placeholder for actual implementation
    return len(clauses)
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        graph = generate_d_regular_graph(n, 2)
        if graph is None:
            continue

        literals, clauses = tseitin_formula(graph, n)
        min_rank = min_rank_of_affine_quotient_sheaf(clauses)
        m_complexity = monotone_circuit_complexity(literals, clauses)

        results.append({
            "n": n,
            "min_rank": min_rank,
            "m_complexity": m_complexity
        })

    if not results:
        return {
            "metric_name": "min_rank_vs_m_complexity",
            "metric_value": 0.0,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "No valid graph generated"
        }

    min_ranks = [r["min_rank"] for r in results]
    m_complexities = [r["m_complexity"] for r in results]

    n_max = max(r["n"] for r in results)
    instances_tested = len(results)

    if any(m <= 0 or mr <= 0 for m, mr in zip(m_complexities, min_ranks)):
        return {
            "metric_name": "min_rank_vs_m_complexity",
            "metric_value": 0.0,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": False,
            "counterexample": "Non-positive values found"
        }

    sum_min_ranks = sum(min_ranks)
    sum_m_complexities = sum(m_complexities)
    sum_min_ranks_squared = sum(x**2 for x in min_ranks)
    sum_m_complexities_squared = sum(x**2 for x in m_complexities)
    sum_min_rank_m_complexity = sum(a * b for a, b in zip(min_ranks, m_complexities))

    n = len(results)
    r_squared = (n * sum_min_rank_m_complexity - sum_min_ranks * sum_m_complexities) ** 2
    r_squared /= ((n * sum_min_ranks_squared - sum_min_ranks**2) * (n * sum_m_complexities_squared - sum_m_complexities**2))

```



The test code uses placeholders for the actual implementation of ‘min\_rank\_of\_affine\_quotient\_sheaf’ and ‘monotone\_circuit\_complexity’, which are crucial components of the conjecture. Without a proper implementation, the test cannot accurately measure the minimal rank of the affine quotient sheaf or the monotone circuit complexity.

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code uses placeholders for critical components of the conjecture, making it impossible to accurately measure the minimal rank of the affine quotient sheaf or the monotone circuit complexity. This invalidates the results. | next: Implement the missing ‘min\_rank\_of\_affine\_quotient\_sheaf’ and ‘monotone\_circuit\_complexity’ functions and conduct a new test that meets all pre-registered criteria.

## 11. Audit log (LLM calls)

**Total LLM calls:** 10

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 22773        |
| 2  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 10362        |
| 3  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 19888        |
| 4  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 22604        |
| 5  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17285        |
| 6  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 28115        |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 22329        |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 21126        |
| 9  | critic          | ollama_remote | glm4:latest      | 0         | 0   | 15021        |
| 10 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 13839        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 193340 ms total latency. Provider mix: {'ollama\_remote': 10}

*(full prompt+response transcripts available in research/audit/7d9763a91a56.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/7d9763a91a56.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/7d9763a91a56.tar.gz (if generated)